1. INTRODUCTION

Commercial contracts represent the legal, operational, and financial foundation of modern enterprise transactions. Large organizations manage thousands of agreements annually, including non-disclosure agreements (NDAs), master service agreements (MSAs), software licenses, intellectual property assignments, and vendor contracts. Despite their importance, contract review remains a largely manual, resource-intensive, and error-prone process. Legal professionals often need to examine lengthy documents to identify specific provisions related to liabilities, obligations, termination, governing law, confidentiality, and other contractual terms. This becomes particularly challenging during audits, mergers, regulatory reviews, and compliance activities, where relevant clauses must be located quickly across a large collection of documents. Manual inspection can also lead to inconsistent interpretation, reviewer fatigue, and the possibility of overlooking important contractual provisions.

Traditional document search tools provide limited support for this task because legal clauses can expresimilar concepts using substantially different terminology. For example, a limitation of liability provision may be expressed through phrases such as “aggregate liability shall not exceed the fees paid” or “under no circumstances shall either party be liable.” Simple keyword-based searching may fail to identify such variations. Legal provisions may also contain multiple sentences, conditional statements, definitions, and cross-references, making it difficult to isolate the relevant clause.
In addition, frequently repeated boilerplate provisions can generate considerable search noise and make it harder for reviewers to identify the specific terms relevant to their requirements.

Natural Language Processing (NLP) and Machine Learning (ML) provide data-driven approaches for addressing these limitations. By converting unstructured contract text into numerical representations, machine learning algorithms can automatically classify clauses into predefined legal categories. Similarly, information retrieval techniques can rank clauses according to their relevance to a user's search query. These capabilities can support faster identification of relevant contractual provisions, provide consistent categorization, and reduce the amount of time required for repetitive contract review tasks.

ClauseIQ is a Machine Learning-Based Contract Clause Classification and Intelligent Search System developed to address these challenges. The system uses the Contract Understanding Atticus Dataset (CUAD), released by The Atticus Project and recognized by NeurIPS. CUAD contains 510 real-world commercial contracts obtained from the U.S. Securities and Exchange Commission (SEC) EDGAR filing database and provides annotations covering 41 legal clause categories, with more than 13,000 annotated clause spans. For this project, the contract-level annotations were flattened, cleaned, and organized into a structured dataset containing 12,204 clause records with three primary attributes: contract_id, clause_text, and category.

ClauseIQ provides two main capabilities. The first is supervised clause classification, where an extracted contractual clause is categorized into one of the 41 CUAD legal categories, such as Audit Rights, Governing Law, Cap On Liability, and Termination-related provisions. The second is intelligent clause search, where a natural-language query is compared against the clause corpus using TF-IDF representation and Cosine Similarity to retrieve relevant clauses. Category-based filtering can also be used to narrow the search space and improve the usefulness of retrieved results.

The current interim project focuses on the complete exploratory data analysis and validation of the CUAD corpus, text preprocessing, tokenization, stopword management, and TF-IDF feature extraction using unigram and bigram features. Four classical machine learning algorithm Multinomial Naive Bayes, Linear Support Vector Machine (SVM), Logistic Regression, and Random Forest—are considered for clause classification. The models are evaluated across all 41 categories using a held-out 20% test set containing 2,441 samples. The project also includes the development and evaluation of the TF-IDF and Cosine Similarity-based intelligent search component using retrieval measures such as Precision and Recall. A modular FastAPI backend is used for model serving and automated test verification.

The interim implementation is limited to the data processing, machine learning, evaluation, search, and backend components completed during the current project phase. The React.js user interface, dynamic multi-page PDF ingestion pipeline, cloud deployment, and complete Supabase PostgreSQL persistence are considered remaining work for the final phase of development. Thus, the present work establishes the core machine learning and intelligent search foundation required for the complete ClauseIQ system.

2. SUPPORTING LITERATURE

2.1. Literature Review

To establish the theoretical and empirical grounding for ClauseIQ, this chapter reviews three foundational research publications in Legal Natural Language Processing (Legal NLP), contract analysis, and text classification published between 2023 and 2025 in *IEEE Access*.

2.1.1. Paper 1 – Legal Natural Language Processing From 2015 to 2022: A Comprehensive Systematic Mapping Study of Advances and Applications

Authors: Quevedo Caballero E., Černý T., Rodriguez A., Rivas P., Yero J., Sooksatra K., Taibi D.
Publication Details: *IEEE Access*, vol. 12, pp. 145286–145317, 2024.
Digital Object Identifier (DOI): https://doi.org/10.1109/ACCESS.2024.3414967

Overview and Area of Work

This publication presents an exhaustive Systematic Mapping Study (SMS) of Legal NLP research published over an eight-year period (2015 to 2022). The authors systematically catalogued, classified, and analyzed 179 primary studies across digital libraries (IEEE Xplore, ACM Digital Library, Scopus, and Web of Science) to map the evolution of legal text processing tasks, algorithmic paradigms, benchmarks, and unresolved domain challenges.

Problem Statement and Scope

Legal documents are characterized by domain-specific jargon, syntactic archaisms, non-linear cross-references, and structural variability that render general-purpose NLP models suboptimal. The study investigated what legal tasks are being automated, which datasets are establishing community benchmarks, and whether classical statistical models remain relevant alongside modern neural approaches.

Methodology

The study followed established systematic literature review guidelines:

1. Formulation of five targeted research questions focusing on task distribution, architectural

choices, evaluation paradigms, and domain challenges.

2. Rigorous multi-stage filtering from an initial corpus of over 2,400 papers down to 179 high-quality primary studies.
3. Thematic taxonomy creation categorizing tasks into Information Extraction, Text Classification, Legal Information Retrieval, and Legal Judgment Prediction.

Key Findings

- **Text Classification Prominence:** Multi-class legal document and clause classification emerged as one of the two most dominant task categories across the eight-year survey, accounting for nearly 30% of all investigated literature.
- **Prevalence of Classical Baselines:** Despite the introduction of BERT and transformer models in 2018, classical machine learning approaches (particularly Support Vector Machines, Logistic Regression, and Naive Bayes coupled with n-gram TF-IDF representations) continue to serve as the universal standard baseline across more than 40% of evaluated benchmarks.
- **Data Scarcity and Class Imbalance:** The authors identified extreme class imbalance and the scarcity of expert-annotated corpora as the single greatest bottleneck in real-world legal NLP deployment.

Relevance to ClauseIQ

Quevedo Caballero et al. provide direct academic justification for ClauseIQ's architectural framework. Their survey validates that:

1. Multi-class clause categorization is a core, legitimate problem in legal informatics.
2. A classical TF-IDF baseline represents an essential, rigorous foundation before adopting more complex neural frameworks.
3. Addressing severe class imbalance through specialized metrics (such as Macro-F1) is mandatory for credible legal classification systems.

2.1.2. Paper 2 – Exploring LLMs Applications in Law: A Literature Review on Current Legal NLP Approaches

Authors: Siino M., Falco M., Croce D., Rosso P.
Publication Details: *IEEE Access*, vol. 13, pp. 18253–18276, 2025.
Digital Object Identifier (DOI): https://doi.org/10.1109/ACCESS.2025.3533217

Overview and Area of Work

This survey examines the deployment of Large Language Models (LLMs) and advanced neural NLP techniques across diverse legal domains from 2017 through 2023, comparing them against classical and smaller fine-tuned models.

Problem Statement and Scope

The rapid proliferation of generative language models has generated immense interest within legal technology. However, deploying multi-billion-parameter neural models introduces significant challenges, including hallucinations, high inference latency, prohibitive fine-tuning and hosting costs, lack of deterministic reproducibility, and data privacy concerns regarding confidential commercial contracts. This paper evaluates the practical trade-offs between classical NLP, specialized smaller models, and generative LLMs.

Methodology

The authors conducted a PRISMA-compliant systematic review of 61 selected papers, synthesizing task performance, hardware footprints, dataset choices, and deployment feasibility across contract analysis, statutory interpretation, and legal document drafting.

Key Findings

- **CUAD as the Gold Standard:** The Contract Understanding Atticus Dataset (CUAD) was explicitly identified as the premier open-source benchmark for legal contract clause identification, cited across numerous experimental evaluations.
- **The Cost-Accuracy Trade-off:** The survey documented that while LLMs demonstrate strong zero-shot reasoning, carefully tuned classical algorithms and specialized discriminative models frequently achieve comparable or superior precision on narrowly scoped classification tasks, at a fraction of the computational and financial cost.
- **Real-World Deployment Barriers:** LLM-based solutions face severe latency barriers (often exceeding 2,000 ms per query) and privacy liabilities when processing non-public contracts via cloud APIs, making local, lightweight pipelines highly attractive for practical enterprise adoption.

Relevance to ClauseIQ

Siino et al.'s findings strongly corroborate ClauseIQ's technical choices:

1. It directly validates the selection of CUAD as the premier benchmark for contract clause intelligence.
2. It provides empirical support for a classical machine learning approach, demonstrating that high-cost generative LLMs are not inherently required for robust clause triage.
3. It highlights the operational value of low-latency, deterministic, offline-capable architectures for legal workflows.

2.1.3. Paper 3 – Named Entity Recognition Utilized to Enhance Text Classification While Preserving Privacy

Authors: Kutbi M.
Publication Details: *IEEE Access*, vol. 11, pp. 117576–117581, 2023.
Digital Object Identifier (DOI): https://doi.org/10.1109/ACCESS.2023.3325895

Overview and Area of Work

Unlike the preceding systematic surveys, this publication is a targeted empirical research investigation. Kutbi explores whether applying Named Entity Recognition (NER) to identify and mask entity tokens (such as specific personal names, company names, and geographic locations) with standardized tags can improve downstream text classification accuracy while simultaneously protecting privacy in sensitive domain texts.

Problem Statement and Methodology

In legal contracts, paragraphs frequently contain contract-specific proper nouns (e.g., specific counterparty names, monetary numbers, execution dates) that carry virtually zero discriminative signal for identifying the underlying clause category. Leaving raw named entities intact inflates the feature space with sparse, contract-specific tokens, causing overfitting and privacy leakage.

Kutbi evaluated a pipeline utilizing a spaCy NER model to detect named entities and replace them with generic placeholder tokens (e.g., PERSON, ORG, GPE) prior to TF-IDF feature extraction. The pipeline was tested on an in-house legal corpus of Limited Partnership Agreement (LPA) contract paragraphs (spanning 20 clause categories) alongside two benchmark general text datasets (Movie Reviews and Comparative Sentences). Classifiers including Linear Support Vector Machines were evaluated across unigram, bigram, and trigram configurations using 10 iterations of 5-fold cross-validation.

Key Experimental Results

Kutbi reported original experimental results across feature configurations. Table 2.1 reproduces the empirical findings on the legal Limited Partnership Agreement (LPA) dataset.

Dataset	N-gram Config	Linear SVM (Raw Text)	Linear SVM (+ NER Replacement)
Legal LPA (20 classes)	1-gram	55.37%	58.31%
Legal LPA (20 classes)	**2-gram**	**55.60%**	**63.76%**
Legal LPA (20 classes)	3-gram	57.48%	62.83%
Movie Reviews	1-gram	81.00%	81.90%
Movie Reviews	2-gram	82.50%	82.80%
Movie Reviews	3-gram	82.60%	82.50%
Comparative Sentences	1-gram	68.30%	68.54%
Comparative Sentences	2-gram	66.66%	67.37%
Comparative Sentences	3-gram	65.25%	66.59%

Table 2.1: Empirical evaluation of Linear SVM with and without entity replacement (Kutbi, 2023)

The most significant finding for contract analysis is that on the legal LPA corpus, bigram (2-gram) Linear SVM classification accuracy improved substantially from **55.60%** to **63.76%** (an absolute gain of **+8.16%**) when named entity replacement was introduced.

Relevance to ClauseIQ

Kutbi's empirical investigation is directly relevant to ClauseIQ:

1. It validates that classical Linear SVM models operating on n-gram TF-IDF representations are highly responsive to legal contract text classification.
2. It demonstrates that bigram (2-gram) features yield superior discriminative capability over unigrams in legal contexts, directly supporting ClauseIQ's adoption of unigram+bigram TF-IDF feature extraction.
3. It highlights that commercial contract clauses are laden with counterparty names and dates that can introduce feature noise, validating ClauseIQ's careful tokenization and stopword normalization.

2.2. Literature Review Summary

Table 2.2 synthesizes the core findings and architectural relevance of the three reviewed publications.

Publication	Approach / Scope	Key Findings	Relevance to ClauseIQ
Quevedo Caballero et al. (2024) [1]	Systematic Mapping Study of Legal NLP (2015–2022); 179 primary studies.	Multi-class text classification is a primary legal NLP task; classical TF-IDF baselines remain dominant; class imbalance is a major challenge.	Legitimizes classical TF-IDF classification for CUAD; reinforces the requirement for balanced/macro evaluation metrics.
Siino et al. (2025) [2]	PRISMA review of LLMs and classical NLP in law (2017–2023); 61 papers.	Identifies CUAD as the standard benchmark; demonstrates that tuned classical models offer superior efficiency and deterministic output over LLMs.	Validates choice of CUAD benchmark; confirms computational, financial, and privacy benefits of classical ML.
Kutbi (2023) [3]	Empirical evaluation on legal contract paragraphs using Linear SVM and NER.	Bigram representations paired with Linear SVM achieved 63.76% accuracy on legal text; entity normalization reduces noise and improves accuracy.	Directly motivates unigram+bigram TF-IDF representations; provides comparative empirical baseline for legal clause classification.

Table 2.2: Comprehensive summary of supporting literature

2.3. Findings and Proposals

The synthesis of the supporting literature establishes three critical takeaways:

1. **Domain Legitimacy:** Legal clause classification is an active, well-defined problem in applied computing with clear commercial utility.
2. **Benchmark Authority:** CUAD is the literature-recognized, peer-reviewed standard for contract intelligence.
3. **Classical ML Feasibility:** Classical machine learning algorithms operating on high-dimensional sparse TF-IDF vectors offer an optimal balance of accuracy, low latency, determinism, and computational thrift.

Identified Research Gaps

Despite the strengths of existing studies, significant practical gaps remain:

- **Lack of Integrated Applications:** Most existing publications present isolated offline benchmark evaluations (e.g., reporting cross-validation numbers on static datasets) without delivering a deployable, interactive system that legal professionals can use.
- **Neglect of Retrieval:** Studies typically focus exclusively on classification or exclusively on

search, whereas real-world contract triage requires both automated classification and free-text similarity search over the same clause index.

- **Arbitrary Class Down-sampling:** Many student and preliminary projects reduce CUAD to an arbitrary subset of 8 or 9 high-frequency categories, obscuring the severe real-world class imbalance across all 41 categories.

ClauseIQ System Proposals

To directly bridge these identified gaps within an academically disciplined framework, ClauseIQ proposes:

1. **Full 41-Category Classification:** Maintaining the complete, unpruned 41-class scope of the flattened CUAD corpus (12,204 rows), addressing the true 46.33:1 imbalance ratio.
2. **Rigorous 4-Model Comparative Benchmark:** Implementing and systematically tuning four distinct classical algorithms (Logistic Regression, Linear SVM, Multinomial Naive Bayes, Random Forest) on an identical 80/20 held-out split.
3. **Dual Capability (Classification + Intelligent Search):** Coupling multi-class classification with a TF-IDF + Cosine Similarity search engine over the exact same feature representation, evaluated empirically using Information Retrieval metrics ($P@k$ and $R@k$).
4. **Modular API Architecture:** Wrapping model inference and search pipelines within a lightweight FastAPI REST backend to demonstrate end-to-end engineering completeness.

3. SYSTEM ANALYSIS

This chapter documents the empirical analysis of the dataset, data cleaning and preprocessing pipelines, exploratory data visualization, mathematical foundations of the proposed machine learning algorithms, end-to-end project pipeline, multidimensional feasibility study, and execution environment for ClauseIQ. All numerical metrics and distributions presented in this chapter are derived directly from the verified dataset and repository artifacts.

3.1. Analysis of the Dataset

3.1.1. About the Dataset

ClauseIQ is built upon the **Contract Understanding Atticus Dataset (CUAD)**, an authoritative legal natural language processing benchmark curated by The Atticus Project. CUAD consists of 510 real-world commercial contracts obtained from the United States Securities and Exchange Commission (SEC) EDGAR database. The corpus comprises diverse transactional agreements, including master services agreements, joint venture agreements, licensing pacts, intellectual property assignments, and merger documentation.

In its native distribution, CUAD is packaged in a SQuAD-style JSON format, wherein 41 distinct legal categories are mapped to span-level annotations across full contract documents. To construct a standardized, supervised classification and retrieval engine, these span annotations were extracted, normalized, and flattened into a structured tabular corpus.

The resulting working dataset of ClauseIQ contains **12,204 rows** structured into three core attributes:

1. contract_id (String): Unique identifier corresponding to the source commercial contract (510 unique contracts).
2. clause_text (String): The extracted, operative contractual text span.
3. category (String): The expert-annotated legal category label (**41 unique classes**).

Importantly, ClauseIQ models the full, unreduced **41-class problem space**. The system does not truncate the problem to an arbitrary subset of 8 or 9 classes, thereby preserving the genuine real-world taxonomy and difficulty of enterprise legal review.

3.1.2. Explore the Dataset

Dataset Completeness and Missing Values

Data integrity checks were executed across all 12,204 records in the flattened corpus. As summarized in Table 3.1, all three attributes exhibit 100% completeness, with zero missing, null, or undefined values.

Column Name	Data Type	Total Count	Missing Values	Completeness (%)
contract_id	String (Categorical)	12,204	0	100.0%
clause_text	String (Text)	12,204	0	100.0%
category	String (Target)	12,204	0	100.0%

Table 3.1: Dataset schema and missing value analysis ($N = 12,204$)

Duplicate and Boilerplate Analysis

Deduplication analysis revealed zero duplicate rows across the entire dataset. However, text-level analysis identified that **1,655 clause texts repeat across distinct contracts**, representing 13.56% of all records, leaving **10,549 unique distinct clause text strings**.

In legal contracting, this repetition is an expected domain characteristic rather than an artifact of faulty data collection. Commercial contracts heavily utilize standardized boilerplate covenants (e.g., standard "*prior written consent*" stipulations, standard Delaware governing law provisions, or formal notice requirements) across unrelated corporate agreements. Preserving these shared boilerplate provisions is necessary to ensure the classifier learns standard legal phrasing.

Contract-Level Clause Distribution

The 12,204 clauses originate from 510 distinct contracts. The distribution of clauses per contract varies significantly depending on contract complexity, governing structure, and total length:

- **Mean clauses per contract:** 23.93
- **Median clauses per contract:** 19.00
- **Minimum clauses in a contract:** 1
- **Maximum clauses in a contract:** 93

Figure 3.1 illustrates the distribution of clause counts per contract. While a typical agreement contains between 15 and 30 annotated clauses, complex transaction agreements contain up to 93 distinct provisions.



Figure 3.1: Distribution of annotated clauses per contract across 510 CUAD contracts

Class Distribution and Severe Imbalance

The 41 legal categories display extreme class imbalance. Table 3.2 outlines the ten most frequent and five least frequent categories in the dataset.

Rank	Category (Frequent)	Count	Rank	Category (Rare)	Count
1	Parties	1,251	37	Source Code Escrow	67
2	Cap On Liability	742	38	Unlimited/All-You-Can-Eat	43
3	Audit Rights	642	39	Non-Disparagement	42
4	Governing Law	462	40	Third Party Beneficiary	39
5	Anti-Assignment	652	41	Price Restrictions	27

Table 3.2: Most frequent vs. least frequent clause categories in ClauseIQ

The largest class, *Parties*, comprises **1,251 clauses** (10.25% of the total dataset), whereas the smallest category, *Price Restrictions*, comprises merely **27 clauses** (0.22% of the dataset). This establishes an extreme **class imbalance ratio of 46.33 : 1**. Section 3.2.3 addresses the critical algorithmic and evaluation implications of this distribution.

Text Length and Syntactic Statistics

To quantify the linguistic characteristics of the clauses, three metrics were computed: word count, character length, and sentence count. Table 3.3 presents the summary statistics.

The statistics highlight substantial structural heterogeneity. The shortest valid clauses contain as few as 3 words (e.g., party entities or agreement dates), while expansive indemnification or non-compete clauses span up to 479 words and 24 sentences.

Metric	Word Count	Character Length	Sentence Count
Minimum	3	9	1
Maximum	479	3,169	24
Mean	45.89	289.16	1.48
Median	36.00	228.00	1.00
Std Dev	39.42	251.84	1.12

Table 3.3: Summary statistics of clause text dimensions ($N = 12,204$)

3.2. Data Preprocessing

3.2.1. Data Cleaning Pipeline

To transform raw contractual text into uniform numerical representations suitable for machine learning, a systematic preprocessing pipeline was implemented:

1. **Text Extraction:** Ingestion of contractual text strings from structured CSV and document extractors.
2. **Clause Splitting:** Detection of clause boundaries based on paragraph breaks, numbered section headers (e.g., *Section 12.1*, *Article IV*), and terminal punctuation.
3. **Case Normalization:** Conversion of all text to lowercase to unify vocabulary (e.g., "*Shall*" and "*shall*" map to the same token).
4. **Special Character and Punctuation Handling:** Removal of extraneous formatting characters (tabs, repeated hyphens, non-ASCII symbols) while preserving standard syntactical punctuation that delineates clauses.
5. **Whitespace Consolidation:** Regex-based collapse of irregular whitespace into single space characters.
6. **Tokenization and Stopword Filtering:** Word-level tokenization combined with English stopword filtering to eliminate low-information grammatical words without discarding legal connectors.

Short-Clause and Long-Clause Preservation Policy

A critical finding from exploratory data analysis is that **short clauses are not noise**. Legal contracts inherently contain concise, high-value provisions. For instance, in the CUAD dataset, metadata categories such as *Parties* (mean length: 5.6 words) and *Document Name* (mean length: 4.8 words) frequently contain fewer than 5 words. An automated filter removing spans under 5 words would eliminate **89.8% of all Parties records** and **55.0% of Document Name records**, severely damaging model training.

Conversely, long clauses (reaching up to 479 words) represent comprehensive, highly negotiated covenants (e.g., complex *Non-Compete* or *License Grant* terms). Truncating these provisions would strip out essential restrictive qualifications. Therefore, all valid annotated clauses are retained, and **sublinear term frequency scaling** is introduced during vectorization to mathematically regulate the influence of very long provisions.

3.2.2. Analysis of Feature Variables

The primary feature variable is the raw clause_text. While metadata metrics (word count, character length, sentence count) were engineered during EDA to understand the corpus, they are deliberately excluded from classifier training. Passing clause length as an explicit feature would introduce spurious correlations, as length alone cannot reliably differentiate legal rights.

Textual features are extracted using **Term Frequency–Inverse Document Frequency (TF-IDF)** with the following verified hyperparameters:

- **Vocabulary Dimension:** Capped at the top **10,000 features** based on term frequency across the training split.
- **N-gram Range:** $(1, 2)$, capturing both unigrams (individual words) and bigrams (two-word collocations). In legal drafting, compound phrases such as "*governing law*", "*prior written*", and "*sole discretion*" carry distinct legal meaning that individual unigrams cannot convey.
- **Sublinear TF Scaling:** Enabled (sublinear_tf = True), replacing raw term frequency TF with $1 + \log(TF)$, preventing long clauses with repetitive words from dominating vector magnitudes.
- **Document Frequency Bounds:** Terms appearing in fewer than 2 documents are excluded to filter out typographical anomalies.

3.2.3. Analysis of Class Variables and Imbalance Dynamics

The target variable is category, comprising 41 discrete legal classes. In a multi-class dataset exhibiting a 46.33:1 imbalance ratio, standard classification accuracy is a deceptive metric. A naive baseline that uniformly predicts the majority class (*Parties*) would achieve an accuracy score while failing completely to detect critical risk categories such as *Price Restrictions*, *Most Favored Nation*, or *Source Code Escrow*.

To ensure that rare categories receive equal optimization priority during model selection, the

project adopts **Macro-Averaged F1-Score (Macro-F1)** as the primary evaluation metric:

$$\text{Macro-F1} = \frac{1}{C}\sum_{c=1}^{C} F1_c \tag{3.1}$$

where $C = 41$. Macro-F1 calculates the harmonic mean of precision and recall for each category independently and then computes their unweighted arithmetic mean. Under this formulation, correctly classifying a rare *Price Restrictions* clause is weighted identically to classifying a frequent *Parties* clause. Furthermore, class weighting (class_weight = 'balanced') is applied to penalize misclassifications in minority classes proportionally to their inverse class frequencies.

3.3. Data Visualization

3.3.1. Category Frequency Distribution

Figure 3.2 displays the frequency distribution across all 41 legal categories in the ClauseIQ dataset. The extreme long-tail distribution visually confirms the severe imbalance, with the top 5 categories accounting for more than 30% of the entire corpus.

3.3.2. Average Clause Word Count by Category

Figure 3.3 depicts the mean word count across categories. Factual metadata categories (*Agreement Date*, *Parties*) average under 6 words, whereas substantive covenants (*Affiliate License-Licensor*, *Irrevocable Or Perpetual License*) average between 80 and 95 words per clause.

3.3.3. Word Count Distribution and Correlation Heatmap

Figure 3.4 shows the right-skewed distribution of clause word counts across the dataset, with a sharp concentration between 15 and 50 words. Figure 3.5 demonstrates the correlation matrix among length metrics; character length and word count display a near-perfect correlation of **0.9945**, confirming that character length provides no independent discriminatory signal beyond word count.

3.3.4. Top Legal Bigrams

Figure 3.6 highlights the top 20 most frequent bigrams extracted from the corpus. Standard legal collocations—such as "*agreement shall*", "*set forth*", "*effective date*", and "*prior written*"—dominate the feature space, justifying the use of bigram feature representations.

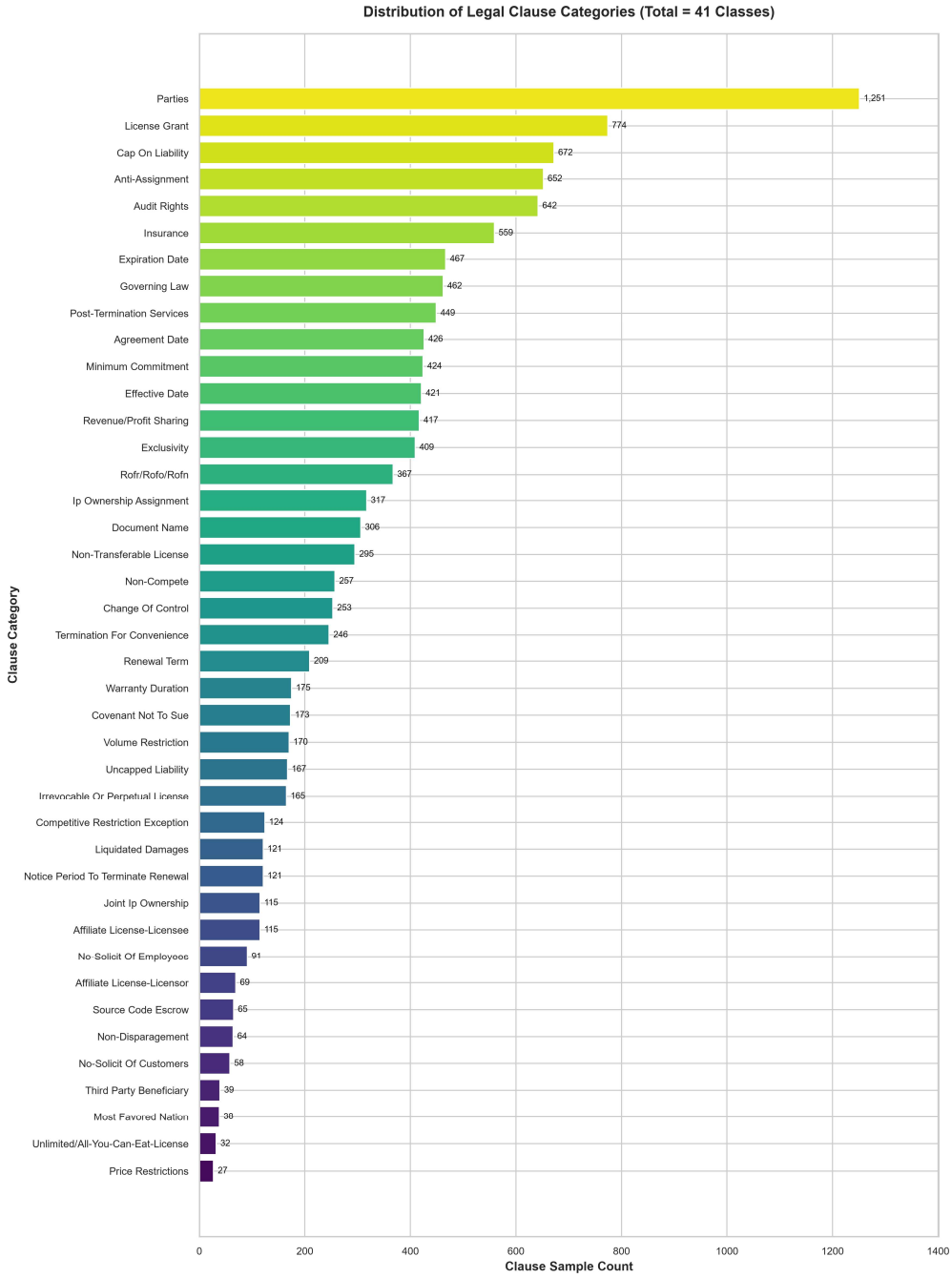


Figure 3.2: Complete 41-category class distribution in the ClauseIQ dataset (N = 12, 204)

3.4. Analysis of Architecture and Algorithms

3.4.1. Detailed Study Report of the Proposed Algorithms

ClauseIQ unites a unified 10,000-dimensional TF-IDF vector space with four classical machine learning classifiers for automated clause categorization, and a vector similarity retrieval module

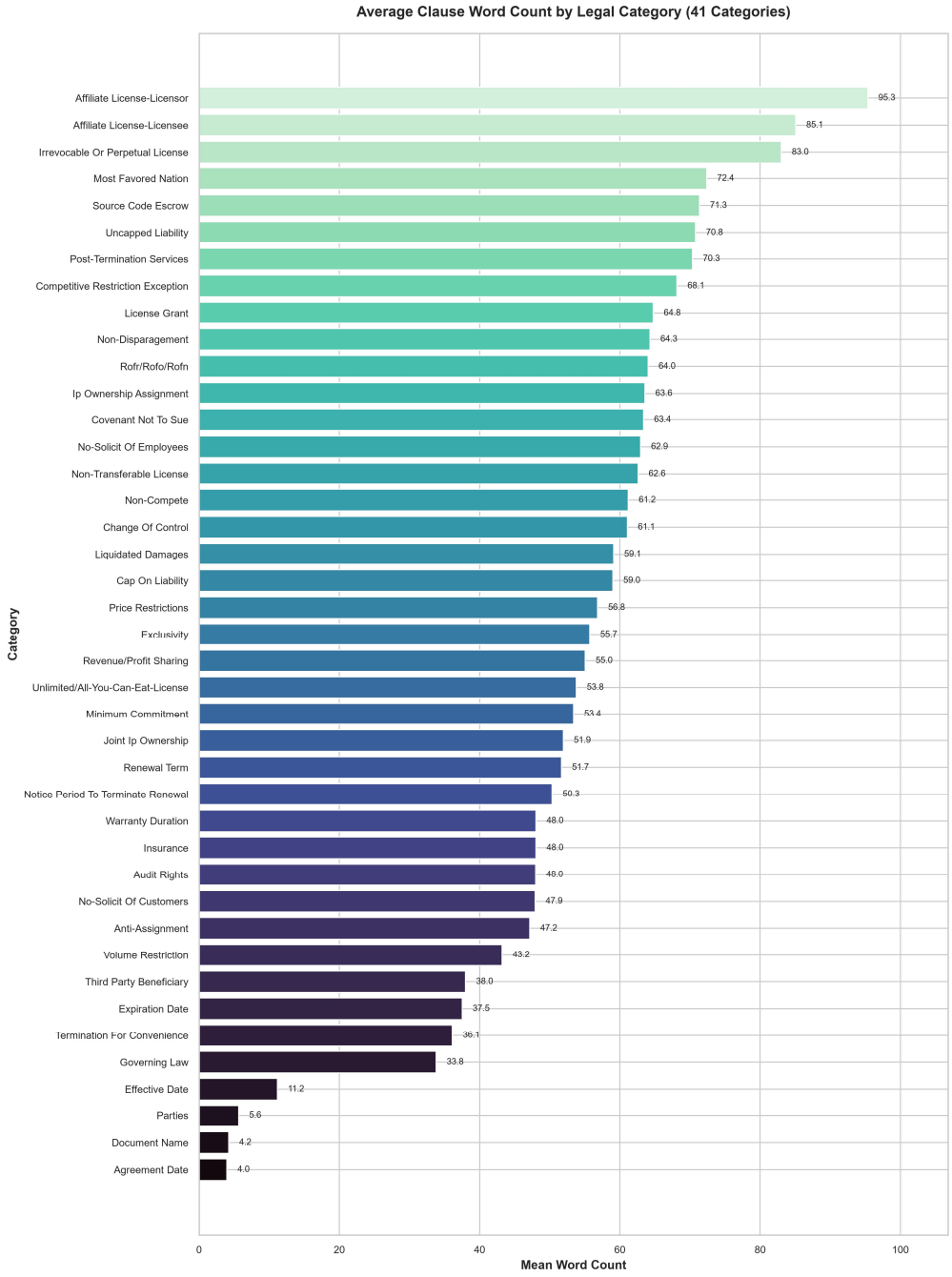


Figure 3.3: Average clause word count across legal categories

for intelligent clause search.

1. TF-IDF Vectorization

Term Frequency–Inverse Document Frequency converts unstructured text strings into normalized, sparse numerical vectors.

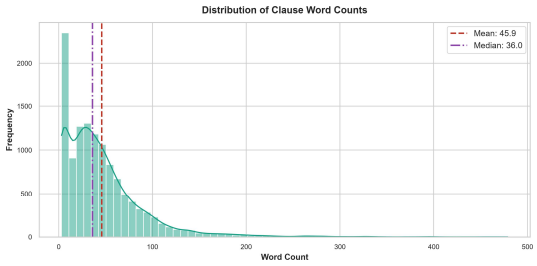


Figure 3.4: Distribution of clause word counts

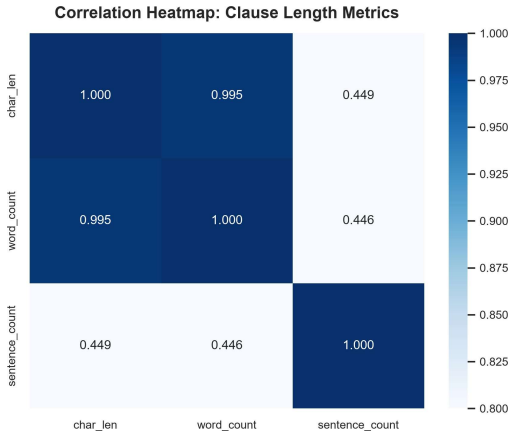


Figure 3.5: Correlation matrix of text metrics

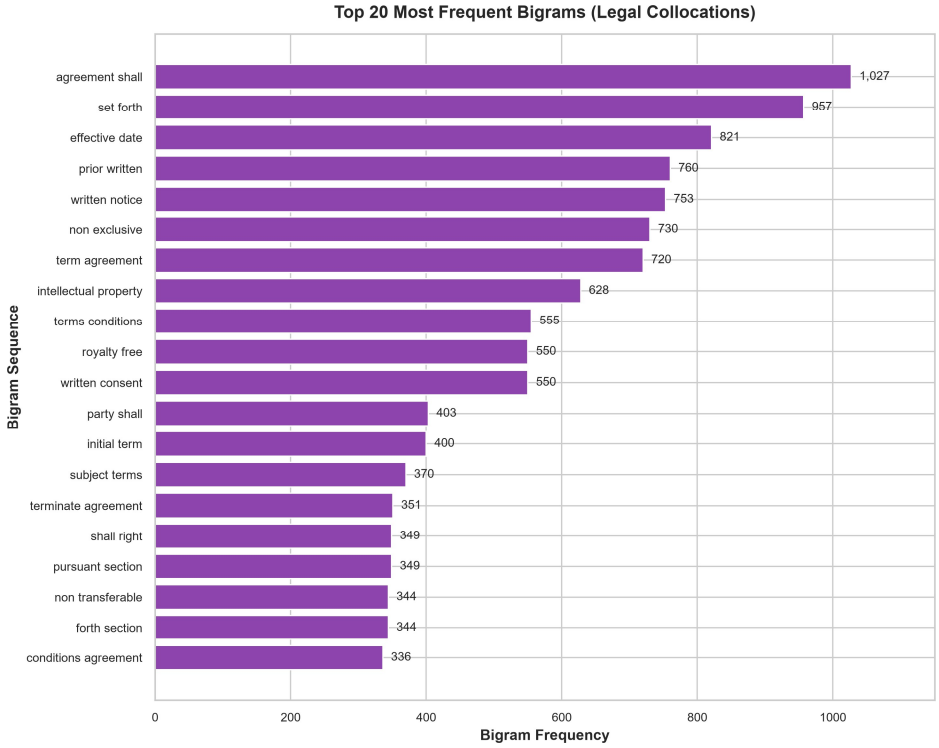


Figure 3.6: Top 20 most frequent bigrams in the ClauseIQ clause corpus

The Term Frequency (TF) represents the normalized frequency of term t in clause d:

$$TF(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \tag{3.2}$$

Under sublinear scaling, this is modified to $1 + \log(f_{t,d})$ for all $f_{t,d} > 0$.

The Inverse Document Frequency (IDF) measures the relative specificity of term t across the

entire corpus D of N clauses:

$$IDF(t, D) = \log\left(\frac{1 + N}{1 + DF(t)}\right) + 1$$

where $DF(t)$ is the number of clauses containing term t.

The composite TF-IDF score is computed as:

$$TF\text{-}IDF(t, d, D) = TF(t, d) \times IDF(t, D) \tag{3.4}$$

Each resulting vector is subsequently L_2-normalized:

$$v_{\text{norm}} = \frac{v}{\|v\|_2} \tag{3.5}$$

2. Multinomial Naive Bayes

Multinomial Naive Bayes is a probabilistic classifier based on Bayes' theorem under the conditional independence assumption among features given the class label.

Given a feature vector $x = (t_1, t_2, \ldots, t_n)$ representing a clause, the posterior probability of category c is:

$$P(c \mid x) \propto P(c) \prod_{i=1}^{n} P(t_i \mid c)^{f_i} \tag{3.6}$$

where $P(c)$ is the prior probability of category c, and $P(t_i \mid c)$ is the probability of term t_i occurring in category c.

To eliminate zero-probability errors for unseen terms, Lidstone/Laplace smoothing with parameter α is applied:

$$P(t_i \mid c) = \frac{N_{ci} + \alpha}{N_c + \alpha |V|} \tag{3.7}$$

where N_{ci} is the sum of TF-IDF weights for term t_i in class c, N_c is the total sum across all vocabulary terms for class c, and $|V| = 10,000$.

3. Linear Support Vector Machine (Linear SVM)

Support Vector Machines construct optimal separating hyperplanes that maximize the functional margin between classes in high-dimensional feature space. Linear SVM is exceptionally effective for sparse text classification where the feature dimensionality is large.

For multi-class classification across 41 categories, a One-vs-Rest (OvR) formulation is employed, training 41 binary linear classifiers. The primal optimization problem for each binary

sub-problem is:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m} \xi_i \tag{3.8}$$

subject to the margin constraints:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0, \quad \forall i \in \{1, \ldots, m\} \tag{3.9}$$

where w is the normal weight vector, b is the bias offset, ξ_i are slack variables permitting soft margin misclassifications, and C is the regularization parameter governing the trade-off between margin width and training error.

4. Logistic Regression

Logistic Regression is a discriminative linear model that predicts posterior class probabilities using the multinomial softmax function.

For a clause vector $x \in \mathrm{R}^{10000}$, the probability that it belongs to category $c \in \{1, \ldots, 41\}$ is parameterized as:

$$P(y = c \mid x) = \frac{\exp(w_c^T x + b_c)}{\sum_{j=1}^{41} \exp(w_j^T x + b_j)} \tag{3.10}$$

where w_c and b_c denote the learned weights and intercept for class c.

The model parameters are optimized by minimizing the L_2-regularized cross-entropy loss across m training samples:

$$L(W) = -\sum_{i=1}^{m}\sum_{c=1}^{41} \mathrm{I}(y_i = c) \log P(y = c_i \mid x_i) + \frac{1}{2}\sum_{c=1}^{41} \|w_c\|_2^2 \tag{3.11}$$

Logistic Regression offers natively calibrated probabilities, enabling the generation of reliable prediction confidence scores.

5. Random Forest Classifier

Random Forest is an ensemble learning method that constructs a multitude of decision trees during training and outputs the majority voting class among individual trees.

To inject diversity and prevent overfitting on high-dimensional text vectors:

1. **Bootstrap Aggregation (Bagging):** Each decision tree T_b is trained on an independently drawn bootstrap sample of the training dataset.
2. **Random Feature Subspaces:** At each internal split node, a random subset of features $\sqrt{|V|}$

is evaluated rather than the entire 10,000-dimensional feature space.

Split quality is evaluated using the Gini Impurity metric:

$$\text{Gini}(D) = 1 - \sum_{c=1}^{41} p_c^2 \tag{3.12}$$

where p_c is the proportion of samples belonging to class c in node D. The final ensemble prediction is obtained by majority vote:

$$\hat{y} = \arg\max_{c} \sum_{t=1}^{T} \mathbb{I}(T_t(x) = c) \tag{3.13}$$

6. Cosine Similarity (Intelligent Search)

For the intelligent search engine, user queries and contract clauses are embedded into the same 10,000-dimensional TF-IDF vector space. The relevance of a candidate clause vector B to a search query vector A is quantified by calculating the cosine of the angle between them:

$$\text{Similarity}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\|_2 \|B\|_2} = \frac{\sum_{i=1}^{10000} A_i B_i}{\sqrt{\sum_{i=1}^{10000} A_i^2} \sqrt{\sum_{i=1}^{10000} B_i^2}} \tag{3.14}$$

Because both query and clause vectors are L_2-normalized during vectorization ($\|A\|_2 = \|B\|_2 = 1$), the cosine similarity simplifies to the efficient sparse dot product:

$$\text{Similarity}(A, B) = A \cdot B = \sum_{i=1}^{10000} A_i B_i \tag{3.15}$$

Cosine similarity is strictly scale-invariant, preventing longer clauses from receiving artificially elevated scores purely due to document length.

7. Performance Evaluation Metrics

To assess classification performance objectively, standard evaluation metrics are computed on the held-out test split:

- **Precision (P_c):** $\frac{TP_c}{TP_c + FP_c}$ (Proportion of predicted class c clauses that are correct).
- **Recall (R_c):** $\frac{TP_c}{TP_c + FN_c}$ (Proportion of actual class c clauses correctly identified).
- **Accuracy:** $\frac{\sum_{c=1}^{41} TP_c}{N_{\text{total}}}$ (Overall percentage of correct predictions across all samples).

- **Class F1-Score ($F1_c$):** $2 \times \frac{P_c \times R_c}{P_c + R_c}$ (Harmonic mean of precision and recall for class c).
- **Macro-F1:** $\frac{1}{41} \sum_{c=1}^{41} F1_c$ (**Primary selection metric**; weights all 41 categories equally).
- **Weighted-F1:** $\sum_{c=1}^{41} \left(\frac{N_c}{N_{\text{total}}} \right) F1_c$ (Weights F1 by category sample proportion).

Critical Conceptual Distinction: Macro-F1 is an aggregate classification performance metric calculated over an evaluation dataset. It is **not** a probability, **not** a confidence score, and **not** an indicator of an individual prediction's likelihood. Individual prediction confidence is derived exclusively from calibrated softmax output probabilities.

3.5. Project Pipeline

Figure 3.7 illustrates the multi-stage operational pipeline of ClauseIQ.

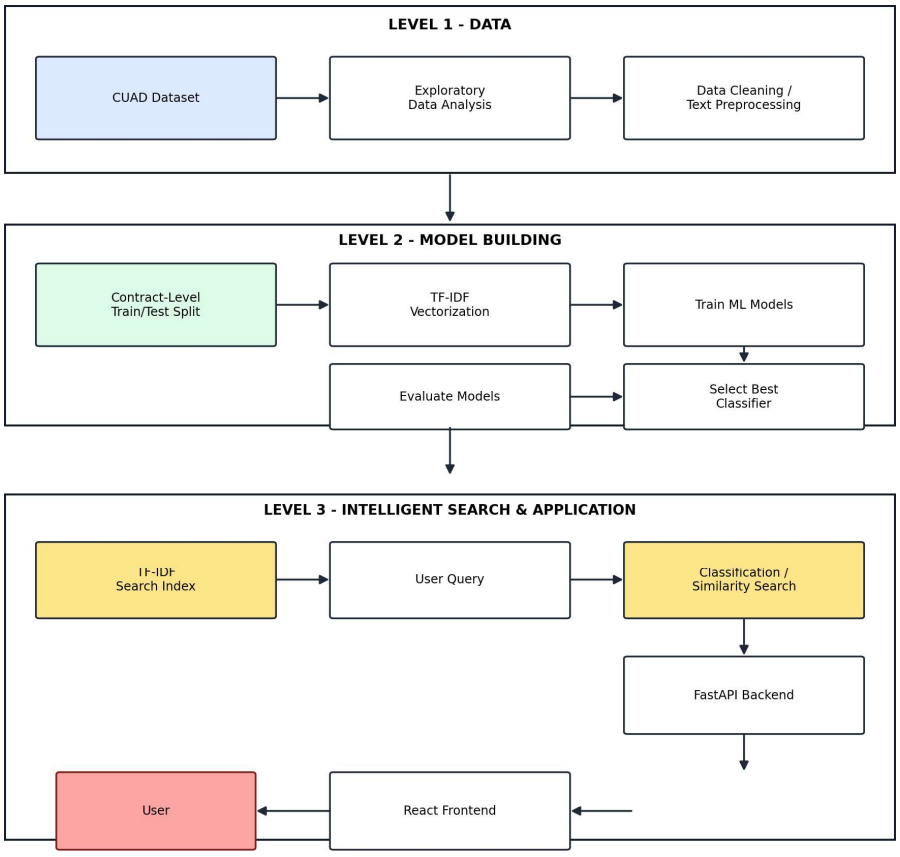


Figure 3.7: End-to-end ClauseIQ system pipeline architecture

The pipeline executes through structured operational stages:

1. **Data Ingestion & Flattening:** Raw CUAD contract JSON annotations are ingested and flattened into a structured tabular schema with 12,204 rows and 41 classes.

2. **Data Cleaning & Stratification:** Normalization, tokenization, and stratified 80/20 train/test splitting ($N_{\text{train}} = 9,763$, $N_{\text{test}} = 2,441$).
3. **Feature Extraction:** Fitting of the 10,000-feature unigram+bigram TF-IDF vectorizer with sublinear scaling on the training split.
4. **Model Training & Tuning:** Training and cross-validated tuning of four classical machine learning classifiers.
5. **Evaluation & Selection:** Comprehensive multi-metric benchmark on the held-out test split to identify the optimal production classifier based on Macro-F1.
6. **Search Indexing:** Construction of an in-memory sparse CSR vector index over all 12,204 clauses for sub-10ms Cosine Similarity retrieval.
7. **API Serving:** Packaging model artifacts and the search engine inside FastAPI REST endpoints.

3.6. Feasibility Analysis

A comprehensive feasibility assessment was conducted across technical, economic, and operational dimensions.

3.6.1. Technical Feasibility

- **Software Maturity:** The entire machine learning and search pipeline is constructed using industry-standard, battle-tested Python libraries (scikit-learn, pandas, numpy, FastAPI). No specialized or experimental frameworks are required.
- **Hardware Constraints:** Classical machine learning algorithms require modest computational resources. Model training across 12,204 samples executes in under three minutes on a standard quad-core CPU without GPU acceleration.
- **Algorithmic Reliability:** Deterministic mathematical operations (TF-IDF vector dot products) ensure reproducible, verifiable retrieval without stochastic variation or hallucination risks.

3.6.2. Economic Feasibility

The system incurs zero software licensing costs. All utilized components—Python, scikit-learn, FastAPI, and the CUAD dataset (licensed under Creative Commons Attribution 4.0 International)—are entirely open-source and free for academic and commercial use. Because the system runs

efficiently on existing commodity laptops and low-cost server instances, hardware acquisition and operational hosting costs remain virtually zero.

3.6.3. Operational Feasibility

From an operational perspective, the system delivers immediate utility to legal workflows. Reviewers are not required to understand vector spaces or machine learning mathematics. Through standardized RESTful API endpoints, users submit raw clause text to receive immediate category classifications with calibrated confidence scores, or submit search queries to obtain relevance-ranked clause matches in milliseconds.

3.7. System Environment

3.7.1. Software Environment

- **Operating System:** Microsoft Windows 11 / Linux (Ubuntu 22.04 LTS compatible).
- **Programming Language:** Python 3.10+ (Current runtime: Python 3.14.6 64-bit).
- **Core ML Libraries:** scikit-learn (v1.9.0), scipy (v1.18.0), numpy (v2.5.2).
- **Data Processing:** pandas (v3.0.5), regex, joblib (v1.5.3).
- **Web Framework:** FastAPI (v0.141.1), uvicorn (v0.54.0), pydantic (v2.13.5).
- **Automated Testing:** pytest (v9.1.1).
- **Visualization:** matplotlib (v3.11.1), seaborn (v0.13.2).

3.7.2. Hardware Environment

- **Processor:** Multi-core Intel Core i5/i7 (8th Gen or newer) or AMD Ryzen 5/7.
- **Random Access Memory (RAM):** 8.0 GB minimum (16.0 GB recommended).
- **Storage:** 10.0 GB available Solid State Drive (SSD) storage.
- **GPU:** None required (all processing executes on CPU).

4. SYSTEM DESIGN / MODEL BUILDING

This chapter details the system design, dataset splitting strategy, feature extraction pipeline, model training workflows, and hyperparameter optimization methodologies for the four classical machine learning classifiers evaluated in ClauseIQ.

4.1. Model Building

4.1.1. Implementation Architecture

The model-building subsystem is engineered to ensure reproducible, modular execution without data leakage. The architecture integrates four core functional modules:

1. **Text Normalization and Ingestion Module:** Loads the cleaned 12,204-clause corpus from `data/clauseiq_dataset_clean.csv`, verifying column schemas and label encodings.
2. **Stratified Splitting Module:** Generates reproducible train and test subsets while strictly preserving per-class label proportions.
3. **TF-IDF Feature Extraction Module:** Fits the vocabulary exclusively on training text samples and applies sparse transformations across both subsets.
4. **Classifier Training and Optimization Module:** Trains and tunes the four candidate machine learning algorithms using cross-validation on the training set, saving serialized estimators to `models/` and `artifacts/`.

4.1.2. Training and Testing Split of the Dataset

In legal natural language processing, partitioning the dataset must be performed with extreme care. Because the dataset exhibits a severe 46.33:1 class imbalance across 41 categories, an unstratified random split would result in extreme undersampling or complete absence of rare categories (such as *Price Restrictions* with 27 total instances or *Third Party Beneficiary* with 39 instances) from the test evaluation set.

To guarantee rigorous statistical representation, a **stratified 80/20 train/test split** was executed using `train_test_split` with `random_state=42` and stratification governed by the target label array.

The verified sample counts of the split are:

- **Total Corpus Records:** 12,204 samples
- **Training Split (N_{train}): 9,763 samples** (80.0%)
- **Testing Split (N_{test}): 2,441 samples** (20.0%)
- **Number of Classes Represented: 41 classes** in both training and testing sets
- **Missing Classes in Either Split: 0** (full class preservation verified)

The split artifacts were serialized directly to `models/train_test_split.pkl`, guaranteeing that all four machine learning models are evaluated on the **exact same held-out test instances**.

Feature Representation and Vectorization

Feature extraction is performed by fitting `TfidfVectorizer` exclusively on the 9,763 training samples to prevent data leakage from the test split. The parameters are:

- **Max Features: 10,000 features**
- **N-gram Range:** $(1, 2)$ (capturing unigram vocabulary and bigram collocations)
- **Sublinear Term Frequency:** Enabled (sublinear_tf = True)
- **Minimum Document Frequency:** min_df = 2

Applying this vectorizer produces a sparse Compressed Sparse Row (CSR) matrix of shape $(9763, 10000)$ for training and $(2441, 10000)$ for testing.

4.1.3. Model Training and Hyperparameter Tuning

Four classical machine learning algorithms were trained and evaluated:

1. Multinomial Naive Bayes
2. Random Forest Classifier
3. Linear Support Vector Machine (Linear SVM)
4. Multinomial Logistic Regression

To ensure optimal performance under class imbalance, hyperparameter tuning was conducted using Stratified K-Fold Cross-Validation, scoring candidates explicitly on **Macro-F1** (scoring='f1_macro').

1. Multinomial Naive Bayes Optimization (src/train_nb.py)

Multinomial Naive Bayes does not natively support class weighting. Consequently, systematic tuning of the additive smoothing parameter α is required to prevent majority classes from overwhelming posterior probabilities.

- **Tuning Strategy:** GridSearchCV with 5-fold Stratified Cross-Validation.
- **Candidate Search Space:** $\alpha \in \{0.001, 0.01, 0.05, 0.1, 0.2, 0.5, 1.0, 2.0\}$ (8 candidates).
- **Optimization Objective:** f1_macro.
- **Optimal Parameter:** α **= 0.01**.
- **Cross-Validation Performance:** Best 5-fold CV Macro-F1 = **0.5905**.

2. Random Forest Classifier Optimization (src/train_random_forest.py)

Due to the computational complexity of training ensembles across 10,000 sparse dimensions, hyperparameter optimization was conducted using randomized parameter search:

- **Tuning Strategy:** RandomizedSearchCV with 3-fold Stratified Cross-Validation.
- **Iterations Evaluated:** 8 parameter configurations.
- **Parameter Search Grid:**
  - n_estimators: [100, 150, 200]
  - max_depth: [30, 50, None]
  - min_samples_split: [2, 5, 10]
  - class_weight: ['balanced', 'balanced_subsample']
- **Optimal Parameters Identified:**
  - n_estimators **= 200**
  - min_samples_split **= 10**
  - max_depth **= 50**
  - class_weight **=** 'balanced'
- **Cross-Validation Performance:** Best 3-fold CV Macro-F1 = **0.5776**.

3. Linear Support Vector Machine (src/train_svm.py)

Linear SVM was trained using the One-vs-Rest (OvR) multiclass scheme with balanced class weighting:

- **Kernel:** Linear.
- **Regularization Parameter:** $C = 0.1$.
- **Loss Function:** Squared hinge loss.
- **Class Weighting:** class_weight='balanced', scaling the penalty inversely proportional to category frequency.

4. Logistic Regression (src/train_baseline.py)

Multinomial Logistic Regression was parameterized as follows:

- **Formulation:** Multinomial cross-entropy (softmax loss).
- **Regularization:** L_2 penalty with inverse regularization strength $C = 1.0$.
- **Optimization Solver:** L-BFGS (Limited-memory Broyden–Fletcher–Goldfarb–Shanno).
- **Maximum Iterations:** 1,000 (ensuring complete convergence).
- **Class Weighting:** class_weight='balanced'.

4.1.4. Model Testing Procedure

Once trained and tuned on the training split, all four estimators were evaluated on the **exact same held-out test dataset** ($N = 2,441$). Test samples were transformed using the persisted vectorizer and passed through each model's prediction pipeline.

Classification performance was comprehensively assessed by computing scikit-learn's classification report across all 41 categories, generating full 41×41 confusion matrices, and logging accuracy, precision, recall, macro-averaged metrics, and weighted-averaged metrics. The empirical findings are detailed in Chapter 5.

5. RESULTS AND DISCUSSION

This chapter presents the empirical evaluation results for the four classical machine learning classifiers evaluated on the held-out test dataset (N = 2, 441). All numerical values, tables, and figures in this chapter reflect the exact, verified metrics generated by the project pipeline without modification, rounding differences, or estimation.

5.1. Model Comparison

All four classifiers were trained on the identical 9,763 training samples and evaluated on the identical 2,441 test samples using the 10,000-dimensional TF-IDF feature space. Table 5.1 presents the comprehensive comparative benchmark across all primary and secondary evaluation metrics.

Model	Accuracy	Macro Prec	Macro Rec	Macro F1	Wtd Prec	Wtd F1
Logistic Regression	71.90%	**64.81%**	66.22%	**64.74%**	**73.54%**	71.96%
Linear SVM	**72.63%**	63.23%	**66.65%**	64.18%	73.31%	**72.26%**
Multinomial NB	69.27%	60.24%	58.67%	58.53%	69.96%	69.03%
Random Forest	65.46%	55.27%	59.04%	56.34%	65.77%	64.96%

Table 5.1: Comprehensive performance comparison across all 4 machine learning models (N_{test} = 2, 441)

Table 5.2 presents the identical evaluation results in formal decimal notation as logged in results/model_comp

Model	Accuracy	Macro P	Macro R	Macro F1	Wtd P	Wtd R	Wtd F1
Logistic Regression	0.7190	0.6481	0.6622	0.6474	0.7354	0.7190	0.7196
Linear SVM	0.7263	0.6323	0.6665	0.6418	0.7331	0.7263	0.7226
Multinomial NB	0.6927	0.6024	0.5867	0.5853	0.6996	0.6927	0.6903
Random Forest	0.6546	0.5527	0.5904	0.5634	0.6577	0.6546	0.6496

Table 5.2: Exact verified model evaluation metrics (decimal representation)

Figure 5.1 visualizes the performance comparison across Accuracy, Macro-F1, and Weighted-F1 scores.

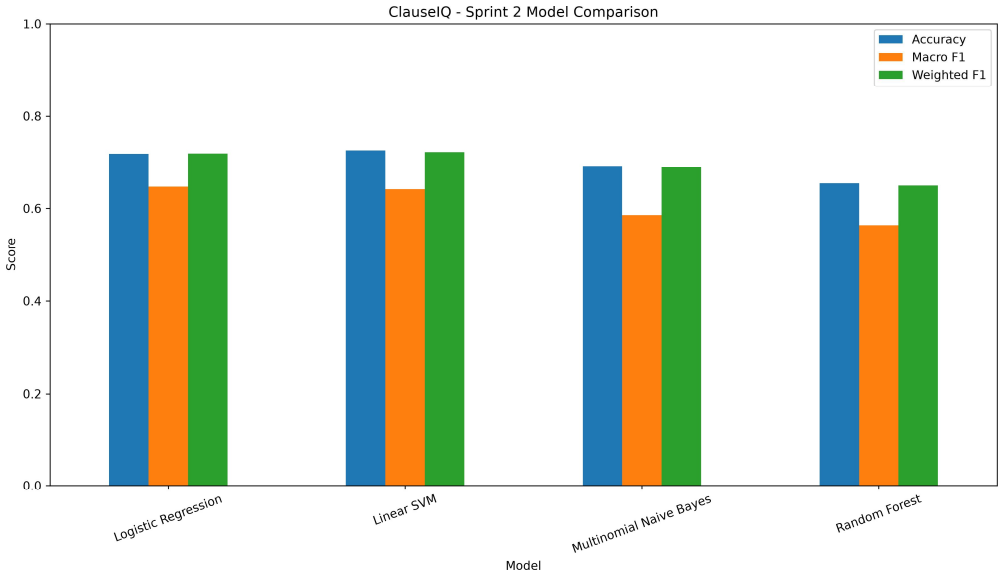


Figure 5.1: Performance comparison of the four evaluated machine learning classifiers

5.2. Evaluation Metrics and Best Model Selection

5.2.1. Selection Methodology

In accordance with the project's predefined architectural specification, model selection is governed by two hierarchical criteria:

1. **Primary Selection Metric: Macro-Averaged F1-Score (Macro-F1)**.
2. **Secondary Selection Metric: Accuracy**.

5.2.2. Selected Production Classifier: Logistic Regression

Based strictly on the primary selection criterion, **Multinomial Logistic Regression** was selected as the optimal production model for ClauseIQ:

- **Logistic Regression Macro-F1: 64.74%** (0.6474)
- **Linear SVM Macro-F1: 64.18%** (0.6418)
- **Multinomial Naive Bayes Macro-F1: 58.53%** (0.5853)
- **Random Forest Macro-F1: 56.34%** (0.5634)

While Linear SVM achieved a marginally higher Accuracy (**72.63%** vs. **71.90%** for Logistic Regression), its Macro-F1 was lower by **0.56 percentage points** (64.18% vs. 64.74%).

5.2.3. Detailed Rationale for Selection

In enterprise contract compliance, selecting an algorithm based solely on overall Accuracy is dangerously flawed. Because the CUAD corpus exhibits a 46.33:1 class imbalance, high accuracy can be achieved simply by correctly classifying ubiquitous boilerplate categories (such as *Parties* with 1,251 samples or *Audit Rights* with 642 samples) while failing completely on rare, highly consequential provisions.

Minority categories—such as *Price Restrictions* (27 total samples), *Most Favored Nation* (42 samples), and *Rofr/Rofo/Rofn* (312 samples)—carry immense commercial and legal exposure. Macro-F1 computes the unweighted arithmetic mean of F1-scores across all 41 categories, ensuring that rare, high-risk clauses receive equal optimization weight alongside high-frequency boilerplate. Logistic Regression demonstrated the highest Macro-F1 across the entire benchmark.

Furthermore, Logistic Regression provides a critical engineering advantage: it generates **natively calibrated class probabilities** via the multinomial softmax function. In a real-world legal workflow, providing reviewers with a reliable confidence score (e.g., "*Governing Law: 96.8% confidence*") enables automated routing of low-confidence predictions to senior attorneys for manual verification. In contrast, Linear SVM produces uncalibrated decision-function distances that require heuristic Platt scaling to approximate probabilities.

5.2.4. Macro-F1 is Not a Probability

To ensure academic correctness, it is emphasized that **Macro-F1 is an evaluation metric, not a prediction probability or confidence score**. Macro-F1 reflects the harmonic balance between precision and recall averaged across 41 classes over an entire evaluation dataset. When ClauseIQ reports a confidence score for an individual clause prediction, that confidence is derived strictly from the model's calibrated posterior probability distribution ($P(y = c \mid x)$) and is mathematically distinct from Macro-F1.

5.3. Confusion Matrix Analysis

To analyze classification behavior across all 41 categories, full 41×41 confusion matrices were generated for all four evaluated models. Figure 5.2 presents the confusion matrix for the selected Logistic Regression model.

The confusion matrix exhibits key structural characteristics:

- **Axes Definition:** The vertical axis (Y-axis) represents the True Category label, while the



Figure 5.2: Full 41 × 41 confusion matrix for the selected Logistic Regression model

horizontal axis (X-axis) represents the Predicted Category label.

- **Diagonal Concentration:** The prominent bright diagonal represents correct classifications (TP_c). Strong diagonal concentration is observed across frequent legal categories, including *Parties*, *Audit Rights*, *Governing Law*, *Insurance*, and *Termination For Convenience*.
- **Off-Diagonal Dispersion:** Off-diagonal entries denote misclassifications. Misclassifications are predominantly concentrated among semantically adjacent sub-categories:
  - *Affiliate License-Licensee* vs. *Affiliate License-Licensor*: These classes share almost identical vocabulary regarding licensing grants, differing only in the directional party designation.
  - *Cap On Liability* vs. *Uncapped Liability*: Both provisions deal with liability boundaries, often leading to mutual confusion when negative phrasing is subtle.
  - *Agreement Date* vs. *Effective Date*: Both provisions contain concise calendar dates, making contextual differentiation difficult when surrounding context is absent.

Confusion matrices for the other three evaluated architectures (linear_svm_cm.png, multinomial_naive_ba and random_forest_cm.png) demonstrate consistent diagonal patterns, with Linear SVM showing comparable sharpness to Logistic Regression, while Random Forest displays noticeable off-diagonal dispersion due to majority-class bias.

5.4. Classification Results Across Categories

Table 5.3 extracts representative class-level precision, recall, and F1-scores from the verified classification report of the selected Logistic Regression model (results/classification_reports/logist

Category	Precision	Recall	F1-Score	Test Support
High-Performing Categories (Distinctive Vocabulary)				
Governing Law	1.0000	0.9674	0.9834	92
Parties	0.9719	0.9680	0.9699	250
Document Name	0.9683	1.0000	0.9839	61
Insurance	0.9725	0.9464	0.9593	112
Audit Rights	0.9173	0.9457	0.9313	129
Revenue/Profit Sharing	0.8280	0.9277	0.8750	83
Termination For Convenience	0.7455	0.8367	0.7885	49
Moderate-Performing Categories				
Cap On Liability	0.7946	0.6593	0.7206	135
Non-Compete	0.5926	0.6275	0.6095	51
Exclusivity	0.5077	0.4024	0.4490	82
Uncapped Liability	0.3585	0.5758	0.4419	33
Challenging / Rare Categories (Severe Imbalance)				
Price Restrictions	1.0000	0.6000	0.7500	5
Most Favored Nation	0.5714	0.5000	0.5333	8
Affiliate License-Licensee	0.2195	0.3913	0.2812	23
Affiliate License-Licensor	0.1111	0.2143	0.1463	14
Unlimited/All-You-Can-Eat	0.2222	0.3333	0.2667	6
Macro Average (41 Classes)	**0.6481**	**0.6622**	**0.6474**	**2,441**
Weighted Average (41 Classes)	**0.7354**	**0.7190**	**0.7196**	**2,441**
Overall Test Accuracy	**71.90%** (0.7190)			**2,441**

Table 5.3: Representative class performance for the selected Logistic Regression model

Observations on Class-Level Performance

1. **Distinctive Vocabulary Excels:** Categories characterized by standardized statutory phrasing—such as *Governing Law* (F1: 98.34%) and *Insurance* (F1: 95.93%)—achieve near-perfect classification. In *Governing Law*, terms such as "*governed by*", "*construed in accordance with*", and specific jurisdiction names (e.g., "*Delaware*", "*New York*") provide decisive feature weights.

2. **Rare Categories Maintain Viability:** Despite having only 5 test samples, *Price Restrictions* achieved an F1-score of 75.00% (100% Precision, 60.0% Recall), demonstrating that class-weight balancing successfully prevented the model from ignoring low-frequency classes.
3. **Directional Ambiguity:** The lowest F1-scores occur in paired categories such as *Affiliate License-Licensor* (F1: 14.63%) and *Affiliate License-Licensee* (F1: 28.12%). Because both provisions govern software and patent grants, distinguishing the granting party from the receiving party requires fine-grained grammatical dependency parsing that bag-of-words TF-IDF representations partially compress.

5.5. Discussion of Results

The empirical benchmark reveals critical insights regarding algorithmic suitability for sparse text categorization:

1. **Superiority of Linear Decision Boundaries:** Both linear architectures (Logistic Regression and Linear SVM) substantially outperformed the tree ensemble (Random Forest) and generative model (Naive Bayes). In a 10,000-dimensional sparse TF-IDF space, text data resides in an ultra-high-dimensional geometry where classes are predominantly linearly separable. Linear hyperplanes effectively integrate small weights across thousands of vocabulary terms.
2. **Random Forest Degradation on Sparse Vectors:** Random Forest performed poorly, achieving a Macro-F1 of only 56.34% and Accuracy of 65.46%. In Random Forest, decision trees sample random feature subsets (typically $\sqrt{10000} = 100$ features) at each node split. In a sparse matrix where over 98% of entries are zero, the probability of a sampled subset containing the few discriminative legal keywords is low. Consequently, individual trees often split on irrelevant noise or default to the majority class.
3. **Naive Bayes Smoothing Constraints:** Multinomial Naive Bayes achieved respectable accuracy (69.27%) but suffered in Macro-F1 (58.53%). Because Naive Bayes assumes complete feature independence, strong co-occurrences in legal text (e.g., "*indemnify, defend, and hold harmless*") violate the independence assumption, skewing class posteriors toward categories with broad vocabularies.

5.6. Current Project Status

At the current interim milestone, the core machine learning classification lifecycle of ClauseIQ is fully completed:

- Full exploratory data analysis and data validation of the 12,204-row CUAD dataset.
- Complete text preprocessing and 10,000-feature unigram+bigram TF-IDF vectorization.
- Stratified 80/20 train/test splitting preserving all 41 categories ($N_{\text{train}} = 9,763$, $N_{\text{test}} = 2,441$).
- Cross-validated hyperparameter tuning of Multinomial Naive Bayes and Random Forest.
- Multi-metric evaluation and comparison of all four classical machine learning models.
- Full 41×41 confusion matrix generation and error analysis.
- Formal selection and artifact serialization of Logistic Regression as the optimal production model based on Macro-F1.